CAN BUS DATA COLLECTION

Data Preparation, Intrusion Detection & Simulation Workflow

TECHNICAL DOCUMENTATION

This document describes the data sources and formats used in the CAN IDS SIM2 work, how captured records are prepared for analysis, and how both stored CAN data and generated virtual traffic pass through the intrusion-detection workflow.

> **Collection context:** The project author reports collecting CAN data in Kargil. The available file names include GEM-labelled dataset assets, and those names alone do not verify that every supplied file originated in Kargil. Data origin should be recorded per capture and confirmed using collection notes or source metadata.

System	CAN IDS SIM2 — CAN Bus Intrusion Detection System
Implementation	Python, pandas, NumPy, scikit-learn / Isolation Forest
Execution modes	Offline dataset evaluation and software-only virtual traffic simulation

1. Overview

CAN IDS SIM2 is a software prototype for examining Controller Area Network (CAN) frames and flagging traffic that appears anomalous according to rule-based checks and a machine-learning baseline. It supports two separate workflows: evaluating stored CSV datasets and generating virtual frames for a repeatable demonstration. Predictions and evaluation summaries are written to local CSV files.

The current implementation is software-only. It does not connect to a physical CAN adapter, vehicle Electronic Control Unit (ECU), or a live vehicle network, and it does not transmit messages onto a real CAN bus.

2. Data Collection and Provenance

The collection location reported by the project author is Kargil. To make this information traceable, each capture should have its own record identifying where and how it was collected. Keep collection location, original data source, file format, and labelling method distinct. A converted or labelled file should retain a reference to its original capture.

Recommended capture record

Field	Information to document
Capture ID	Unique identifier for the recording, linked to the original filename.
Collection location	Kargil only where confirmed for that specific capture; use a site code if appropriate.
Date and conditions	Date/time range, collection duration, and relevant test or driving conditions.
Equipment and software	Capture interface, logger, software version, and configuration, where known.
Labelling method	Label source, rules used, and any manual validation performed.
Sharing authorization	Confirm permission to share the data and remove sensitive details where required.

3. Data Files and Formats

File/component	Purpose
GEM_CAN_Dataset.csv	Labelled CSV sample used for offline evaluation.
GEM_Normal_Driving_Raw.json	Raw normal-driving example in JSON format.
GEM_Normal_Driving_Raw_Labeled.csv	Normal-driving records in labelled CSV form.
GEM_Attack_Scenario_Raw.trc	Saved TRC capture parsed by the conversion utility.
GEM_Attack_Scenario_Raw_Labeled.csv	Labelled attack-scenario CSV used for evaluation.
GEM_CAN_metadata.json	Metadata accompanying the GEM-named data assets.
data_preprocessing.py	Utilities for cleaning and preparing data.
trc_to_csv_converter.py	Local utility for converting TRC records to CSV.

4. Data Preparation and Feature Extraction

Raw capture files may need conversion before they can be evaluated as a table. The TRC conversion utility parses frame records and writes CSV output. Prepared labelled CSVs can be loaded directly in dataset mode. The main detection script normalizes CAN identifiers and payload bytes, derives timing and payload features, and maps available source labels to normal/attack targets where the input schema supports it.

The features used by the current detector include the numeric CAN identifier (can_id_num), data length code (dlc), mean payload-byte value (mean_byte), inter-arrival time (delta), and the first two payload bytes (b0 and b1). These are statistical features, not decoded vehicle-specific signals from a DBC file.

Data path distinction: Dataset mode reads a selected CSV from disk. Demo mode generates virtual frames in code. The demo does not automatically replay the reported Kargil captures or the GEM-named files; stored data is evaluated through the separate dataset mode.

5. Simulation and Detection Workflow

1. Select the input path

Use demo mode to generate virtual CAN traffic, or dataset mode to load a prepared CSV.

2. Prepare frame features

Normalize identifiers, payload values, DLC and timing information into fields expected by the detector.

3. Apply rule checks

Rule logic checks for suspicious conditions such as identifiers outside the normal baseline, CAN ID zero, and unusually short frame intervals, according to the implementation.

4. Apply the ML baseline

An Isolation Forest model is fitted on a normal training portion and identifies records that appear anomalous.

5. Combine decisions

The hybrid decision flags a frame when either the rules or ML component flags it. This can improve detection coverage but can also raise false positives.

6. Evaluate labelled data

Where labels are available, compare predictions with labels and calculate accuracy, precision, recall, F1-score, and confusion-matrix counts.

7. Save results

Write per-frame predictions and summary results to CSV files in the outputs directory.

Workflow at a glance

Stored capture TRC / JSON	**Convert / prepare** Clean and label	**Feature extraction** ID, DLC, payload, timing
Dataset mode Load CSV	**Detection engine** Rules + Isolation Forest	**Results** Predictions + metrics

The virtual demo path begins at the in-code traffic generator and passes generated frames to detection; it does not require a physical CAN connection.

6. How the Components Connect

Connection	What it does	Current status
TRC → converter → CSV	Parses a saved capture into tabular records.	Local conversion utility
Labelled CSV → dataset mode	Loads records, prepares features, and evaluates predictions.	Implemented
Virtual generator → detector	Creates synthetic frame sequences for repeatable demonstrations.	Implemented
Detector → result CSV	Stores per-frame outputs and metrics locally.	Implemented
Physical CAN interface / ECU	Would require separate hardware integration and safety validation.	Not connected

7. Execution Commands

Run these commands from the folder containing can_ids_sim2.py, with the Python environment activated:

```
python can_ids_sim2.py --mode demo
python can_ids_sim2.py --mode dataset --input data/GEM_CAN_Dataset.csv
python can_ids_sim2.py --mode dataset --input data/GEM_Attack_Scenario_Raw_Labeled.csv
python trc_to_csv_converter.py "data/GEM_Attack_Scenario_Raw.trc" --output-dir data
```

The converter's labels are rule-derived and should be reviewed against capture notes. Dataset runs may overwrite a shared summary output file; preserve a copy when comparing multiple experiments.

8. Evaluation and Reporting

The program reports accuracy, precision, recall, F1-score, and confusion-matrix counts for labelled inputs. Review false positives and false negatives alongside these aggregate metrics. Accuracy can be misleading when one class dominates the dataset, and labels generated by heuristics can encode assumptions that affect apparent performance.

The small synthetic demo is useful for demonstrating program flow, but its metrics should not be described as real-world performance. For stronger evaluation, use a representative held-out set, document the train/test split, compare results across attack types, and retain the source and label-generation details for each file.

9. Limitations and Responsible Use

- This is a software-only simulation; no live vehicle network is connected.
- There is no vehicle-specific DBC signal decoding or semantic validation.
- Feature choices and thresholds may not generalize across vehicles or driving conditions.
- Labels generated during conversion may not represent every attack variant.
- Results depend on data quality, class balance, label accuracy, and evaluation split.

If a capture is described as collected in Kargil, retain supporting collection notes for that specific file. Before sharing captured data publicly, verify permission to publish and remove sensitive or identifying information where necessary. Clearly distinguish locally collected data, third-party data, derived labelled records, and synthetic traffic.

10. Summary

The workflow connects stored CAN data and generated virtual traffic to a common local detection process. Saved captures can be converted into CSV and evaluated offline; the demo mode provides repeatable virtual traffic for observing rule-based, ML-based, and hybrid decisions. Traceable provenance and transparent labelling are important for interpreting results accurately.

Scope: technical description of the current software workflow. It is not a claim of validated automotive security or production deployment.